

Отчет по лабораторной 1 «Написание нейросети для классификации изображений»

```
[27]: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

[105]: # =====
# ПАРАМЕТРЫ
# =====
BATCH_SIZE = 32
IMG_SIZE = 100
EPOCHS = 50

# сколько эпох ждём улучшения перед остановкой
PATIENCE = 50

[70]: # =====
# ПРЕОБРАЗОВАНИЯ
# =====
# приводим все картинки к одному размеру + нормализация
transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.5,0.5,0.5],[0.5,0.5,0.5])
])

[71]: # =====
```

Рисунок 1- Импорт библиотек и входные параметры

```
71]: # =====
# ДАННЫЕ
# =====
#train_data = datasets.ImageFolder('train1', transform=transform)
train_data=datasets.ImageFolder(root='train1', transform=transform)
#val_data = datasets.ImageFolder('val1', transform=transform)
val_data=datasets.ImageFolder(root='val1', transform=transform)

train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE)

72]: # =====
```

Рисунок 2- Чтение датасета

```

# МОДЕЛЬ
# =====
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()

        # сверточная часть - извлекает признаки из изображений
        self.conv = nn.Sequential(
            nn.Conv2d(3,16,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(16,32,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32,64,3,padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        # полносвязная часть - принимает решение (класс)
        self.fc = nn.Sequential(
            nn.Linear(64*12*12,128),
            nn.ReLU(),
            nn.Linear(128,2)
        )

    def forward(self,x):
        x = self.conv(x)
        x = x.view(x.size(0),-1)
        return self.fc(x)

```

Рисунок 3 -Архитектура сети

```

model = SimpleCNN()

]: # =====
# НАСТРОЙКИ ОБУЧЕНИЯ
# =====
criterion = nn.CrossEntropyLoss()

```

Рисунок 4- Создание модели

```

optimizer = optim.Adam(model.parameters(), lr=0.0005)

# =====
# ПЕРЕМЕННЫЕ ДЛЯ КОНТРОЛЯ ПЕРЕОБУЧЕНИЯ
# =====
best_accuracy = 0 # лучшая точность
epochs_no_improve = 0 # сколько эпох подряд нет улучшения

```

Рисунок 5 -Параметры обучения

```

# =====
# ОБУЧЕНИЕ + ВАЛИДАЦИЯ
# =====
for epoch in range(EPOCHS):

    # ---- TRAIN ----
    model.train()
    train_loss = 0

    for images, labels in train_loader:
        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        train_loss += loss.item()

    # ---- VALIDATION ----
    # проверяем качество на новых данных
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            outputs = model(images)

            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

# ---- Вывод ----

```

Рисунок 6 — Обучение

```

# ---- Вывод ----
print(f"Epoch {epoch+1}")
print(f"Loss: {train_loss:.4f}")
print(f"Accuracy: {accuracy:.2f}%")
print("-----")

# =====
# КОНТРОЛЬ ПЕРЕОБУЧЕНИЯ
# =====

# если модель стала лучше → сохраняем её
if accuracy > best_accuracy:
    best_accuracy = accuracy
    epochs_no_improve = 0

    torch.save(model.state_dict(), "best_model.pth")
    print("✓ Новая лучшая модель сохранена")

else:
    epochs_no_improve += 1
    print(f"Нет улучшения: {epochs_no_improve}/{PATIENCE}")

# если долго нет улучшений → останавливаем обучение
if epochs_no_improve >= PATIENCE:
    print("Ранняя остановка (early stopping)")
    break

# =====
# СОХРАНЕНИЕ ФИНАЛЬНОЙ МОДЕЛИ
# =====
torch.save(model.state_dict(), "model_finetuned.pth")

```

Рисунок 7- Обработка результатов

```
Loss: 0.0122
Accuracy: 54.29%
-----
Нет улучшения: 39/50
Epoch 45
Loss: 0.0117
Accuracy: 54.29%
-----
Нет улучшения: 40/50
Epoch 46
Loss: 0.0105
Accuracy: 54.29%
-----
Нет улучшения: 41/50
Epoch 47
Loss: 0.0103
Accuracy: 55.71%
-----
Нет улучшения: 42/50
Epoch 48
Loss: 0.0093
Accuracy: 54.29%
-----
Нет улучшения: 43/50
Epoch 49
Loss: 0.0089
Accuracy: 54.29%
-----
Нет улучшения: 44/50
Epoch 50
Loss: 0.0090
Accuracy: 52.86%
-----
Нет улучшения: 45/50
```

Рисунок 8- Результат обучения (метрики)

```
[132]: # =====
# ТЕСТ НА ПАПKE testbl
# =====

import os
from PIL import Image
import torch.nn.functional as F

[133]: print("\n=== Проверка на тестовых изображениях ===")

=== Проверка на тестовых изображениях ===

[134]: # загружаем ЛУЧШУЮ модель
model.load_state_dict(torch.load("model_finetuned.pth"))
model.eval()
```

Рисунок 9- загрузка последних весов для тестирования

```
135]: test_dir = "test"
      classes = train_data.classes

136]: classes

136]: ['cat', 'dog']

137]: for file in os.listdir(test_dir):

      if file.endswith((".jpg", ".png", ".jpeg")):

          path = os.path.join(test_dir, file)

          img = Image.open(path).convert("RGB")
          img = transform(img)
          img = img.unsqueeze(0)

          with torch.no_grad():
              output = model(img)
              probs = F.softmax(output, dim=1)

          confidence, predicted = torch.max(probs, 1)

          print(f"\nФайл: {file}")
          print("Предсказание:", classes[predicted.item()])
          print("Уверенность:", round(confidence.item(), 3))
```

Рисунок 10 — Обработка тестовых изображений

Файл: english_cocker_spaniel_121_jpg.rf.db065f7b515f83644b2fcb7525073d0b.jpg
Предсказание: dog
Уверенность: 1.0

Файл: yorkshire_terrier_106_jpg.rf.d6f3d2d3861cfdb0b43daab736c5c3d7.jpg
Предсказание: cat
Уверенность: 0.583

Файл: Persian_105_jpg.rf.ea4d455c191a11efb7d07b377717618a.jpg
Предсказание: cat
Уверенность: 0.999

Файл: boxer_12_jpg.rf.e900c37bda862f16a7a42b9fd21fa8e3.jpg
Предсказание: cat
Уверенность: 0.735

Файл: english_setter_165_jpg.rf.ece3ff5a48702e310cfd029b9601c718.jpg
Предсказание: dog
Уверенность: 1.0

Файл: american_pit_bull_terrier_134_jpg.rf.fae91eb602b864f71fe5e53b170f8132.jpg
Предсказание: cat
Уверенность: 0.849

Файл: american_pit_bull_terrier_144_jpg.rf.e8ae7c9b81d3361e4129508b8050fea2.jpg
Предсказание: dog
Уверенность: 0.543

Файл: samoyed_100_jpg.rf.e30772b0bbeea1692aef33c667989368.jpg
Предсказание: dog
Уверенность: 1.0

Рисунок 11- Результат предсказания тестовых изображений



Рисунок 12 — Изображение
«english_cocker_spaniel_121_jpg.rf.db065f7b515f83644b2fcb7525073d0b»



Рисунок 13- Изображение «yorkshire_terrier_106_jpg.rf.d6f3d2d3861cfdb0b43daab736c5c3d7»



Рисунок 14- Изображение «Persian_105_jpg.rf.ea4d455c191a11efb7d07b377717618a»



Рисунок 14- Изображение «boxer_12_jpg.rf.e900c37bda862f16a7a42b9fd21fa8e3»



Рисунок 15- Изображение «english_setter_165_jpg.rf.ece3ff5a48702e310cfd029b9601c718»



Рисунок 16- Изображение
«american_pit_bull_terrier_134_jpg.rf.fae91eb602b864f71fe5e53b170f8132»



Рисунок 17- Изображение «samoyed_100_jpg.rf.e30772b0bbeca1692aef33c667989368»

